

MACHINE LEARNING | FASTAPI PROJECT

Intelligent Diabetes Risk Predictor

Architecture, Approach & Technology Stack

End-to-end ML system: data -> model -> REST API -> web UI

Dataset: Pima Indians Diabetes (768 records)

Design Document v1.0

1. Project Overview

The Intelligent Diabetes Risk Predictor is a machine-learning application that estimates a patient's probability of having diabetes from eight routine clinical measurements (glucose, BMI, age, blood pressure, insulin, and others). A trained scikit-learn model is served through a FastAPI backend and consumed by a simple web interface, giving an instant, colour-coded risk read-out.

Problem statement

Early identification of individuals at risk of diabetes enables timely lifestyle intervention. Manual screening is slow and inconsistent. A data-driven predictor provides a fast, repeatable first-pass risk estimate that can support (not replace) clinical judgement.

Objectives

- Train a reliable binary classifier on the Pima Indians Diabetes dataset.
- Expose the model through a clean REST API with input validation and auto-docs.
- Provide a user-friendly web form that returns a risk percentage and risk band.
- Apply sound ML engineering: no data leakage, missing-value handling, model selection.

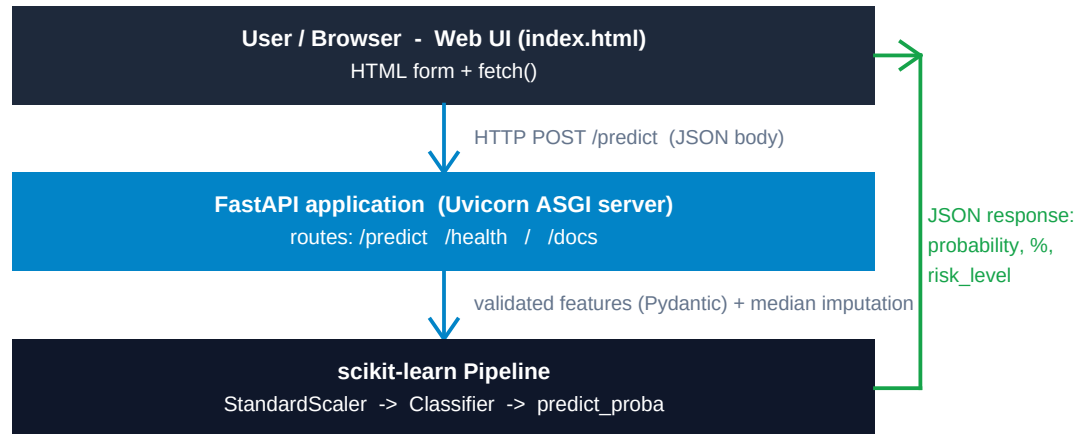
Scope & disclaimer

This is an educational / demonstration project, not a certified medical device. The dataset represents adult female patients of Pima Indian heritage, so predictions outside that population are not clinically validated.

2. System Architecture

The system has two decoupled halves: an OFFLINE training pipeline that produces model artifacts, and an ONLINE serving path that loads those artifacts and answers requests.

A. Serving path (runtime)



B. Training pipeline (offline, run once via train.py)



Because training and serving share the exact same preprocessing (the medians learned at training time are stored in metadata.json and re-applied at inference), the model sees production inputs in precisely the form it was trained on - eliminating train/serve skew.

3. How It Works - Request Lifecycle

- **1. User input** The user fills the web form (glucose, BMI, age, etc.) and clicks Predict. JavaScript packages the values into a JSON object.
- **2. HTTP request** A POST request is sent to /predict with the JSON body and Content-Type: application/json.
- **3. Validation** FastAPI parses the body into a Pydantic PatientData model. Out-of-range or missing fields are rejected automatically with a 422 error and a clear message.
- **4. Preprocessing** Biologically impossible zeros (e.g. Glucose=0) are replaced with the training-set median stored in metadata.json - identical to training.
- **5. Inference** The scikit-learn pipeline scales the features and calls predict_proba to get the probability of the positive (diabetic) class.
- **6. Risk banding** The probability is mapped to a band: Low (<30%), Moderate (30-60%), High (>=60%), and a 0/1 label at a 0.5 threshold.
- **7. Response** A JSON response (probability, risk_percent, risk_level, prediction, model_name) is returned and rendered as a colour-coded result card.

4. Development Approach

The project was built in clear phases, each producing a testable artifact.

- **Phase 1 - Data** Acquire the Pima dataset, inspect distributions, and identify the 'impossible zero' problem in Glucose, BloodPressure, SkinThickness, Insulin and BMI.
- **Phase 2 - Modelling** Split data (stratified 80/20). Impute missing values using medians computed on the TRAINING split only. Standardise features.
- **Phase 3 - Selection** Compare Logistic Regression, Random Forest and Gradient Boosting via 5-fold cross-validated ROC-AUC; keep the best pipeline.
- **Phase 4 - Evaluation** Report accuracy, ROC-AUC and a full classification report on the held-out test set. Persist model.joblib + metadata.json.
- **Phase 5 - API** Wrap the model in FastAPI with Pydantic validation, a /predict endpoint, a /health check, and auto-generated Swagger docs at /docs.
- **Phase 6 - Frontend** Build a lightweight HTML/JS page that posts to the API and shows a colour-coded risk card.
- **Phase 7 - Verify & document** Test low/high/invalid inputs end-to-end; write the README and this design document.

Engineering principles applied

- No data leakage - imputation statistics learned on training data only.
- Reproducibility - fixed random_state; artifacts versioned as files.
- Train/serve parity - one shared preprocessing definition in metadata.json.
- Separation of concerns - training, model, API and UI are independent modules.

5. Technology Stack

Layer	Technology	Purpose
Language	Python 3.13	Rich ML + web ecosystem; single language end to end.
ML library	scikit-learn	Pipelines, imputation, scaling, model selection, metrics.
Data handling	pandas / NumPy	Loading the CSV and vectorised feature processing.
Persistence	joblib + JSON	Serialise the fitted pipeline and preprocessing metadata.
API framework	FastAPI	Async, type-safe REST API with automatic OpenAPI/Swagger docs.
Validation	Pydantic v2	Declarative request schemas with range checks and errors.
Web server	Uvicorn	High-performance ASGI server to run FastAPI.
Frontend	HTML + CSS + JS	Zero-build single page using the native fetch() API.
Dataset	Pima Indians Diabetes	768 labelled clinical records, 8 features.

Model performance (this build)

- Selected model: Logistic Regression (best cross-validated ROC-AUC).
- Cross-validated ROC-AUC ~ 0.84; held-out test ROC-AUC ~ 0.81; accuracy ~ 0.71.

6. API Endpoints & Future Work

REST endpoints

- **POST /predict** Accepts patient JSON, returns probability, risk_percent, risk_level and prediction.
- **GET /health** Liveness check; returns model name and evaluation metrics.
- **GET /** Serves the web UI (index.html).
- **GET /docs** Interactive Swagger UI (auto-generated by FastAPI).

Future enhancements

- Explainability: add SHAP values so each prediction shows which factors drove the risk.
- Containerisation: Dockerfile + docker-compose for one-command, portable deployment.
- Model registry & retraining: schedule periodic retraining and track model versions.
- Persistence & auth: store predictions in a database and secure the API with API keys.
- Testing & CI: pytest suite for the API plus a GitHub Actions pipeline.
- Monitoring: log prediction distributions to detect data drift over time.

This document describes the design of an educational machine-learning project. It is not intended for clinical use.